

Produit et technique

Plateforme de recherche, réservation et gestion hôtelière
Architecture AWS définie par CDK TypeScript

Document	Valeur
Version	1.0
Date	5 septembre 2026
Statut	Référence de conception — code validé localement, non déployé
Région cible	us-east-1
Portée	Produit, logiciel, infrastructure, exploitation et feuille de route

Position de lecture

Le document décrit l'état du code et des templates synthétisés. Il ne constitue ni une preuve de déploiement, ni une mesure de disponibilité, de latence ou de délivrabilité en production.

SYNTHÈSE

Résumé exécutif et contrôle du document

RoomHop est un prototype fonctionnel avancé de plateforme hôtelière. Le voyageur recherche une disponibilité réelle, réserve et annule ses séjours. Le propriétaire d'hôtel soumet une demande, attend une validation puis publie et administre son offre. Les événements métier alimentent les e-mails et l'analytique.

L'architecture cible sépare le frontend, la recherche, les réservations, les notifications et l'analytique. Elle combine CloudFront, WAF, API Gateway, Cognito, ECS/Fargate, RDS MySQL, OpenSearch, DMS, EventBridge, SQS, Lambda, SES, S3, Glue, Athena, Metabase et un pipeline CodePipeline. Treize stacks CDK décrivent 270 ressources CloudFormation dans la dernière synthèse active.

Décisions intentionnelles conservées

RDS MySQL reste Single-AZ. Le VPC possède deux subnets privés isolés dans deux AZ, tandis que Search, Reservation et Metabase démarrent chacun avec desiredCount = 1. Ces choix réduisent le coût d'une démonstration mais limitent la haute disponibilité.

Dimension	Conclusion
Maturité	Architecture de référence testée localement ; pas une plateforme exploitée.
Cohérence métier	Transactions MySQL et verrous FOR UPDATE protègent le stock.
Résilience	Files, DLQ, archive et fallback existent ; plusieurs composants sont mono-instance.
Sécurité	Bon socle, mais ouverture par défaut des listeners ALB et validation des claims à renforcer.
Notification	Chaîne codée et testée ; réception SES non vérifiable sans déploiement/configuration.
Priorité	Fermer les listeners, renforcer validations, observabilité et atomicité des événements.

Public visé : sponsor produit, architecte, développeur, DevOps, sécurité, QA et futur exploitant.

NAVIGATION

Table des matières

Pagination conçue pour une lecture dans OnlyOffice ; les numéros seront vérifiés sur le rendu final.

§	Section	Page
1	Vision, acteurs et périmètre	4
2	Exigences fonctionnelles — voyageur	5
3	Exigences fonctionnelles — partenaire et administration	6
4	Parcours voyageur	7
5	Parcours partenaire et notification	8
6	Règles métier	9
7	Exigences non fonctionnelles	10
8	Architecture d'ensemble	11
9	Décisions et invariants d'architecture	12
10	Réseau, edge et exposition	13
11	Compute, routage API et conteneurs	14
12	Données relationnelles et migrations	15
13	Recherche, DMS et fallback MySQL	16
14	Cohérence des stocks et idempotence	17
15	Événements, files et e-mails	18
16	Analytique et Metabase	19
17	Identité, autorisation et protection	20
18	Frontend, UX et accessibilité	21
19	Infrastructure as Code et CI/CD	22
20	Observabilité et opérations	23
21	Validation et préparation au déploiement	24
22	Modèle de coût théorique	25
23	Évaluation Well-Architected — méthode et synthèse	26
24	Well-Architected — opérations et sécurité	27
25	Well-Architected — fiabilité et performance	28
26	Well-Architected — coûts et durabilité	29
27	Feuille de route d'amélioration	30
28	Registre des risques	31
29	Traçabilité et couverture de tests	32
30	Glossaire et références	33

1

Vision, acteurs et périmètre

Vision. Réunir découverte, disponibilité, réservation fiable, gestion partenaire et données décisionnelles dans une expérience cohérente, soutenue par une architecture AWS privée et événementielle.

Acteur	Objectif	Capacités principales
Visiteur	Comparer une offre	Accueil, recherche publique, prix, disponibilités, équipements.
Voyageur	Réserver et gérer	Compte, réservation, historique, annulation.
Demandeur partenaire	Obtenir un accès	Formulaire professionnel, suivi de statut, accusé e-mail.
Partenaire approuvé	Publier l'offre	Création d'hôtel, chambres, tarifs, stocks, modifications API.
SuperAdmin	Contrôler l'accès	Lecture, approbation ou rejet des demandes.
Analyste	Comprendre l'activité	S3, Glue, Athena et Metabase.
Exploitant	Livrer et maintenir	CDK, pipeline, logs, traces, DLQ et audit.

Inclus. Recherche, authentification, réservation, annulation, onboarding partenaire, revue, publication d'un établissement, notifications, analytics, développement local et architecture AWS.

Hors périmètre actuel. Paiement en ligne, remboursement, multi-devises, PMS/channel manager, avis clients, fidélité, application mobile, facturation PDF serveur et tableaux de bord Metabase préconfigurés.

Niveau de maturité

Prototype fonctionnel avancé / architecture de référence. Les composants sont codés et testés localement, mais aucune ressource AWS n'est déployée et aucun SLA ne peut être affirmé.

Preuves dans Le dépôt : `README.md` ; `infra/README.md` ; `infra/bin/app.ts` ; `hotel-ui/src/App.jsx`

2

Exigences fonctionnelles — voyageur

ID	Exigence	État
RF-G01	Rechercher par destination, arrivée, départ et voyageurs.	Implémenté/testé
RF-G02	Filtrer par prix et équipements.	Implémenté ; étoiles partiel
RF-G03	Exiger tarif et stock pour chaque nuit.	Implémenté/testé
RF-G04	Utiliser OpenSearch puis MySQL en cas d'erreur.	Implémenté/testé
RF-G05	Créer, confirmer et ouvrir une session Cognito.	Implémenté ; AWS non testé
RF-G06	Réserver sous identité JWT, jamais avec un guest_id client.	Implémenté/testé
RF-G07	Calculer le montant côté serveur.	Implémenté/testé
RF-G08	Éviter doublons et surréservation.	Implémenté ; intégration réelle à ajouter
RF-G09	Lister et lire uniquement ses séjours.	Implémenté/testé
RF-G10	Annuler et restituer le stock une seule fois.	Implémenté/testé

Écarts fonctionnels visibles

Le filtre d'étoiles est affiché mais pas appliqué. Les dates initiales de la page d'accueil sont figées en juillet 2026. L'UI réserve une chambre, ne stocke pas les demandes spéciales et le récapitulatif peut sous-estimer une période à tarifs variables.

Critère d'acceptation central : deux requêtes simultanées sur la dernière unité disponible ne doivent jamais créer deux réservations confirmées.

Preuves dans Le dépôt : hotel-ui/src/pages/HomePage.jsx ; SearchResultsPage.jsx ; BookingPage.jsx ; ReservationsPage.jsx ; services/search-service ; services/reservation-service

3

Exigences fonctionnelles — partenaire et administration

ID	Exigence	État
RF-P01	Soumettre une demande Hotel Property Owner authentifiée.	Implémenté/testé
RF-P02	Ignorer toute identité Cognito fournie par le navigateur.	Implémenté/testé
RF-P03	Attribuer HotelPartnerPending et exposer le statut.	Implémenté
RF-P04	Envoyer l'accusé PartnerApplicationSubmitted.	Code/test ; SES non vérifié
RF-P05	Limiter la décision au groupe SuperAdmin.	Implémenté
RF-P06	Approuver/rejeter et synchroniser groupes + attribut.	Implémenté/testé service
RF-P07	Réserver la publication aux partenaires approuvés.	Implémenté/testé
RF-P08	Créer propriété, chambres, tarifs et stocks atomiquement.	Implémenté/testé service
RF-P09	Limiter chaque partenaire à ses propres hôtels.	Implémenté/testé
RF-P10	Administrer l'offre après création.	API présente ; UI partielle

- Champs obligatoires : identité professionnelle, société, identifiant fiscal, contact, siège, portefeuille estimé, ville principale et site HTTP/HTTPS.
- La création complète initialise 365 jours de tarifs et de stocks et lie l'hôtel au partenaire.
- L'interface de revue SuperAdmin suppose encore que l'identifiant de la demande est connu ; aucune file visuelle globale n'est finalisée.
- L'upload d'images et les écrans complets d'ajustement des prix/stocks restent à développer.

Preuves dans Le dépôt : `hotel-ui/src/pages/PartnerOnboardingPage.jsx` ; `PropertyRegistrationPage.jsx` ; `SuperAdminReviewPage.jsx` ; `services/reservation-service/src/services/adminService.js`

4

Parcours voyageur

Recherche → authentication → réservation → gestion

1. Le visiteur saisit destination, dates, voyageurs et prix ; les critères sont transportés dans l'URL.
2. Le service contrôle les paramètres et recherche la disponibilité sur toute la période.
3. OpenSearch répond en priorité. Une exception ou une absence de configuration bascule sur MySQL.
4. Les résultats sont regroupés par hôtel ; les équipements peuvent être filtrés dans le navigateur.
5. Le voyageur choisit un type de chambre. Une route protégée l'envoie vers la connexion si nécessaire.
6. Le formulaire est prérempli à partir de Cognito et crée une clé d'idempotence.
7. Le serveur rattache ou crée le client à partir du sub Cognito, verrouille chaque nuit, calcule le montant puis commit.
8. Après le commit, BookingConfirmed est publié ; l'interface confirme et la notification est mise en file.
9. La page Mes réservations filtre par identité Cognito. Une annulation autorisée restitue le stock dans une transaction.

Règle d'annulation actuelle

La fenêtre est de 72 heures après la création de la réservation. Ce n'est pas une règle « jusqu'à trois jours avant l'arrivée ». Le libellé produit doit éviter toute ambiguïté.

Réponse	Sens
200/201	Recherche ou réservation réussie ; replay idempotent possible.
400	Entrée invalide ou incomplète.
401/403	Absence d'identité ou droit insuffisant.
404	Réservation/ressource inaccessible au compte.
409	Stock insuffisant, période incomplète ou état incompatible.

5

Parcours partenaire et notification

10. Un utilisateur authentifié ouvre Manage Hotel Property et complète le formulaire professionnel.
11. Le frontend construit un payload métier sans cognitoSub ; le serveur réutilise exclusivement l'identité authentifiée.
12. MySQL crée ou remet la demande à PENDING, puis Cognito place l'utilisateur dans HotelPartnerPending.
13. PartnerApplicationSubmitted est envoyé au bus EventBridge.
14. La règle partenaire alimente la file SQS de notification ; Lambda choisit le modèle et appelle SES.
15. Le demandeur consulte une page d'attente présentant la référence et l'adresse utilisée.
16. Un SuperAdmin approuve ou rejette. Les groupes Cognito et custom:partner_status sont mis à jour.
17. PartnerApplicationReviewed déclenche le message de décision.
18. Après approbation, le partenaire publie un établissement puis n'administre que les hôtels qui lui sont liés.

Pourquoi un e-mail peut manquer après un futur déploiement

SES exige une identité expéditrice vérifiée ; en sandbox, le destinataire doit aussi être vérifié. Il faut contrôler le paramètre SesSenderEmail, la région, les logs Lambda, la notification DLQ et l'archive EventBridge. Le code et les tests ne prouvent pas une délivrabilité réelle.

Preuves dans Le dépôt : `infra/lib/events-stack.ts` ; `services/Lambda/notification-handler/index.js` ; `services/reservation-service/src/services/adminService.js`

6

Règles métier

Domaine	Règle
Séjour	Intervalle semi-ouvert : arrivée incluse, départ exclu ; arrivée < départ.
Recherche	1 à 50 voyageurs ; capacité du type de chambre $\geq$ voyageurs.
Disponibilité	Une ligne de tarif et une ligne de stock doivent exister pour chaque nuit.
Stock	Disponible = minimum de total_inventory – total_reserved sur la période.
Montant	Somme des nightly_rate $\times$ room_count ; monnaie actuelle EUR.
Réservation	Création en CONFIRMED ; annulation en CANCELLED, jamais suppression physique.
Identité	sub et e-mail Cognito sont les données de confiance pour le voyageur.
Idempotence	Une même clé ne doit produire qu'une réservation métier.
Annulation	Propriétaire uniquement ; au plus 72 h après created_at ; stock restitué une fois.
Partenaire	PENDING, APPROVED ou REJECTED ; décision réservée à SuperAdmin.
Propriété	Partenaire approuvé et propriétaire du lien ; suppression logique via deleted_at.
Initialisation	Une publication de propriété crée 365 jours de tarif et d'inventaire.
Paieement	Pay at hotel ; aucun paiement ni remboursement en ligne.

À durcir

La validation de création de réservation vérifie surtout la présence des champs. Les types, bornes, room_count entier et positif, cohérence des dates et contraintes SQL doivent être renforcés.

Preuves dans Le dépôt : database/migration_v2.sql ;
services/reservation-service/src/middleWare/validate.js ;
services/reservation-service/src/services/reservationService.js

7

Exigences non fonctionnelles

Qualité	Mécanisme attendu	Appréciation
Sécurité	JWT, groupes, propriété des données, WAF, secrets et chiffrement.	Bon socle ; chemin interne à durcir
Cohérence	Transactions InnoDB et FOR UPDATE.	Fort pour stock ; tests réels à ajouter
Résilience	SQS/DLQ, archive, backup RDS et fallback.	Partielle, composants mono-instance
Performance	CloudFront, OpenSearch, index SQL, autoscaling 1–4.	Prometteur ; aucun SLO mesuré
Scalabilité	Services stateless et asynchronisme.	Applicatif scalable ; données limitées
Observabilité	Logs, Container Insights, CloudTrail, daemon X-Ray.	Alarmes/dashboard/instrumentation absents
Maintenabilité	Services séparés, 13 stacks, tests et pipeline.	Bonne modularité
Portabilité	Environnement Docker et cible AWS.	Risque de dérive locale/cloud
Accessibilité	Labels, focus, responsive, reduced motion sur onboarding.	Audit global absent
Confidentialité	Chiffrement et réseau privé.	Politique PII/RGPD à définir
Disponibilité	Deux AZ réseau.	RDS, OpenSearch et tâches initiales non HA
Coût	Sans NAT, petites tailles, desiredCount 1.	Endpoints privés dominants

Objectifs à formaliser

Disponibilité cible, latence p50/p95, taux d'erreur, RTO, RPO, retard DMS, taux de fallback, taux de livraison e-mail et coût par réservation.

8

Architecture d'ensemble

Architecture logique RoomHop — état défini par le code

Flux synchrones en bleu, réplication en violet, événements en orange

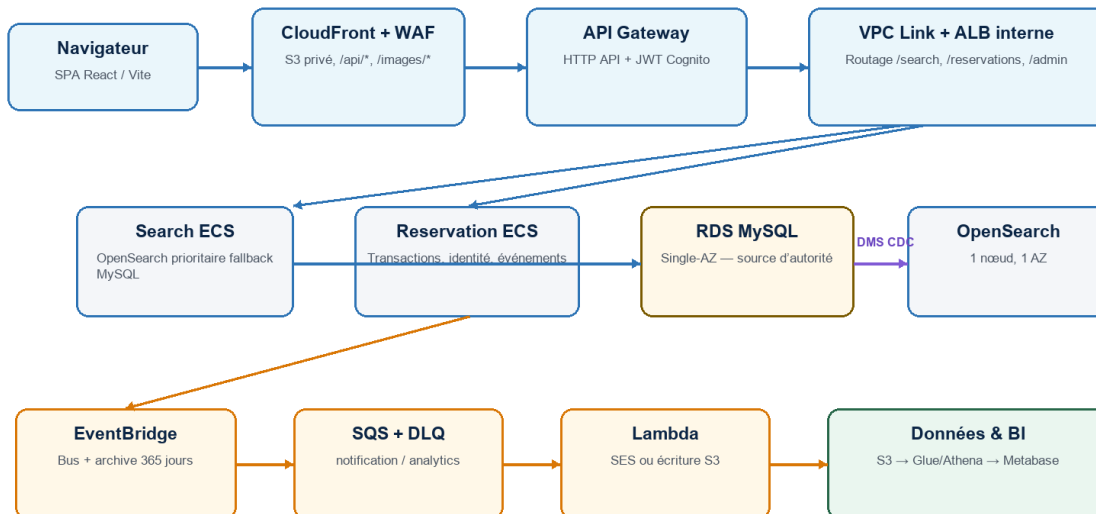


Figure 1 — Architecture logique définie par les stacks CDK.

La source d'autorité transactionnelle est MySQL. OpenSearch optimise la lecture ; DMS réplique les données en full-load puis CDC. Les commandes de réservation restent synchrones, tandis que notification et analytique sont découplées par événements et files. Le frontend et les images sont privés dans S3 et servis via CloudFront.

État réel

Le diagramme représente un end-state codé. Aucun domaine, cluster, bucket, pipeline ou base présenté ici n'existe encore dans AWS.

Décisions et invariants d'architecture

ID	Décision	Justification	Conséquence
ADR-01	RDS MySQL Single-AZ	Coût et simplicité de démonstration.	Point de panne accepté ; backup/restore impératif.
ADR-02	Deux subnets isolés dans deux AZ	Segmentation et préparation multi-AZ.	Pas d'accès internet direct.
ADR-03	desiredCount initial = 1	Limiter le coût d'une courte exécution.	Une tâche perdue crée une interruption jusqu'au remplacement.
ADR-04	Aucun NAT Gateway	Réduire coût et surface sortante.	11 endpoints interface × 2 AZ.
ADR-05	OpenSearch + fallback MySQL	Recherche rapide sans abandonner l'autorité relationnelle.	Fraîcheur DMS et résultat vide à surveiller.
ADR-06	EventBridge + SQS	Découpler producteur, notification et analytics.	Cohérence commit/événement à compléter.
ADR-07	CDK multi-stack	Séparation des responsabilités et synthèse vérifiable.	Ordonnancement et configuration plus complexes.
ADR-08	Metabase conteneurisé	BI flexible sur Athena.	Service toujours actif, auth séparée.

Invariants protégés par tests

Deux subnets, zéro NAT, RDS MultiAZ=false, desiredCount=1 pour Search/Reservation, un subnet OpenSearch et Lambdas notification/analytics hors VPC.

Preuves dans Le dépôt : `infra/lib/config.ts` ; `infra/test/architecture.test.ts`

10

Réseau, edge et exposition

Topologie réseau et domaines de défaillance

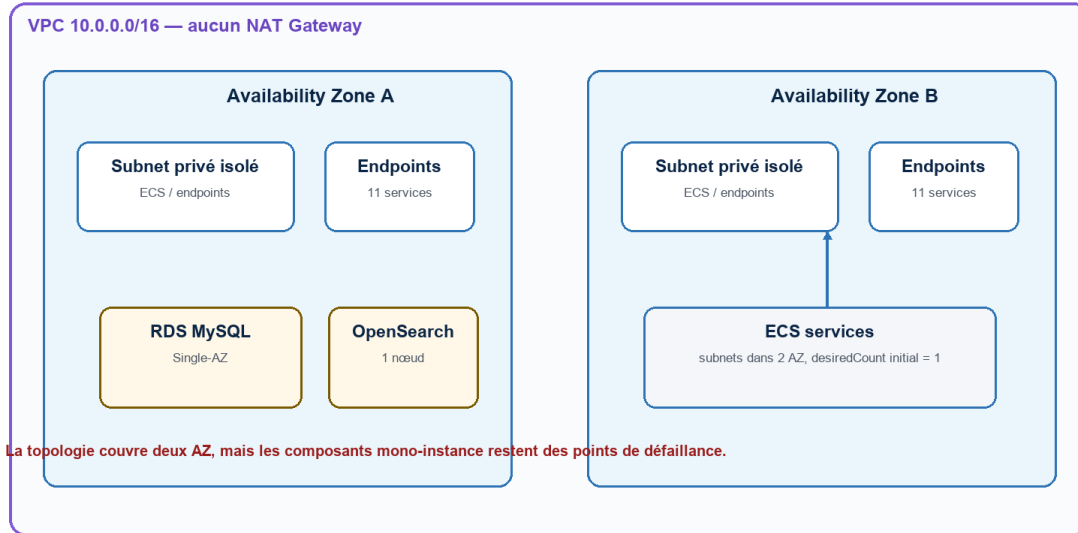


Figure 2 — Répartition réseau sur deux AZ et composants mono-instance.

Contrôle	Configuration
Front door	CloudFront + WAF ; buckets S3 privés via OAC. Les URLs API Gateway restent directement joignables et peuvent contourner le WAF.
API	HTTP API, JWT Cognito, VPC Link vers ALB interne.
Sortie privée	Endpoint S3 Gateway + 11 types d'endpoints Interface dans 2 AZ.
Security groups	VPC Link→ALB ; ALB→ECS 3000 ; ECS/Lambda/DMS→données.
Écart critique	Listeners ALB open par défaut : règles 0.0.0.0/0 sur 80/8080 dans le template.

Correction recommandée avant déploiement

Créer les listeners avec open: false et ne conserver que l'ingress depuis le security group VPC Link. Restreindre aussi l'accès direct aux endpoints API Gateway, limiter CORS aux origines attendues et poser CSP/HSTS sur les réponses edge.

11

Compute, routage API et conteneurs

Composant	Taille	Instances	Rôle
Search ECS	0,5 vCPU / 1 Gio	1 initial ; autoscaling 1–4	Recherche, OpenSearch et fallback MySQL.
Reservation ECS	0,5 vCPU / 1 Gio	1 initial ; autoscaling 1–4	Réservation, annulation et admin partenaire.
Metabase ECS	1 vCPU / 2 Gio	1	Exploration BI via Athena.
ALB interne	Listeners 80 et 8080	managé	Routage chemins application et Metabase.
ECR	4 repositories	managé	Search, Reservation, X-Ray et Metabase.

Routage. CloudFront envoie `/api/*` vers API Gateway. Les routes publiques de recherche et protégées de réservation/administration sont intégrées à l'ALB par VPC Link. Un second HTTP API et une seconde distribution exposent Metabase.

Conteneurs. Les task definitions utilisent des Docker assets pour le premier déploiement ; le pipeline pousse ensuite les images versionnées dans ECR et met à jour les services ECS.

Santé. Chaque service expose `/health`. L'ALB utilise des target groups dédiés. Les services sont stateless ; les sessions et données restent dans Cognito/MySQL.

Nuance X-Ray

Un sidecar daemon X-Ray est provisionné, mais aucun `aws-xray-sdk` n'est trouvé dans les services. Le document ne considère donc pas le traçage distribué comme opérationnel tant que l'instrumentation n'est pas ajoutée.

Preuves dans Le dépôt : `infra/Lib/compute-stack.ts` ; `infra/Lib/api-stack.ts` ; `infra/Lib/metabase-stack.ts` ; `infra/docker`

12

Données relationnelles et migrations

Entité	Responsabilité
hotel	Métadonnées, étoiles, localisation et suppression logique.
room_type	Capacité et équipements par catégorie.
room_type_rate	Tarif quotidien unique par hôtel/type/date.
room_type_inventory	Stock et réservations quotidiennes ; compteur cohérent.
guest	Profil voyageur et liaison unique au sub Cognito.
reservation	Séjour, statut, quantité, montant et clé d'idempotence.
hotel_images	Références d'images et image principale.
hotel_administrators	Demande, société, statut et identité Cognito.
hotel_admin_properties	Relation d'appartenance partenaire-hôtel.

RDS. MySQL 8.0.46, db.t3.medium, Single-AZ, 50 Gio extensibles à 200 Gio, chiffrement, sauvegarde sept jours, Performance Insights et secret généré.

Migration. Une Lambda Node.js 24 dans les subnets isolés exécute un bootstrap idempotent et des incréments. DELETE est volontairement sans effet. La version de ressource personnalisée est 7.

Binlogs. La procédure mysql.rds_set_configuration fixe la rétention à 24 heures. Cette fenêtre laisse DMS reprendre le CDC après une interruption courte ; elle doit dépasser la durée maximale de panne prévue.

Limite de preuve

Le schéma et les plans de migration sont testés, mais aucune migration n'a été exécutée contre un RDS réel.

Preuves dans Le dépôt : `infra/lib/database-stack.ts` ; `services/Lambda/db-migration/index.js` ; `database/migration_v2.sql`

13

Recherche, DMS et fallback MySQL

Chemin principal. Le service signe les requêtes OpenSearch avec SigV4, applique un timeout de cinq secondes, interroge des index séparés puis assemble hôtels, types, tarifs, stocks et images. La période doit être couverte nuit par nuit.

Réplication. DMS dms.t3.micro, 20 Gio, Single-AZ, exécute full-load-and-cdc depuis MySQL vers OpenSearch pour hotel, room_type, room_type_rate, room_type_inventory et hotel_images.

Fallback. Si l'endpoint n'est pas configuré ou si OpenSearch lève une erreur, une requête SQL équivalente est exécutée sur la source d'autorité. La réponse annonce source: opensearch ou source: mysql.

```
try { results = await searchOpenSearch(criteria); source = 'opensearch'; }
catch (error) { results = await searchMySql(criteria); source = 'mysql'; }
```

Risque	Conséquence	Amélioration
Index incomplet mais réponse vide valide	Aucun fallback ; faux zéro résultat.	Vérifier fraîcheur/compteurs et fallback conditionnel.
Retard DMS > rétention binlog	CDC irrécupérable sans rechargement.	Alarme lag et procédure full reload.
SupportLobs=false avec hotel.description TEXT	Description potentiellement tronquée/ignorée.	Limited LOB ou VARCHAR borné.
Starter seulement onCreate	Task remplacée lors d'un update peut rester STOPPED.	Gérer onUpdate et état idempotent.
Un nœud OpenSearch	Interruption de recherche principale.	Accepter le fallback ou passer multi-nœud en prod.

Preuves dans Le dépôt : `services/search-service/src/opensearch.js` ; `routes/search.js` ; `infra/Lib/dms-stack.ts` ; `infra/Lib/opensearch-stack.ts`

14

Cohérence des stocks et idempotence

Séquence de réservation et cohérence du stock

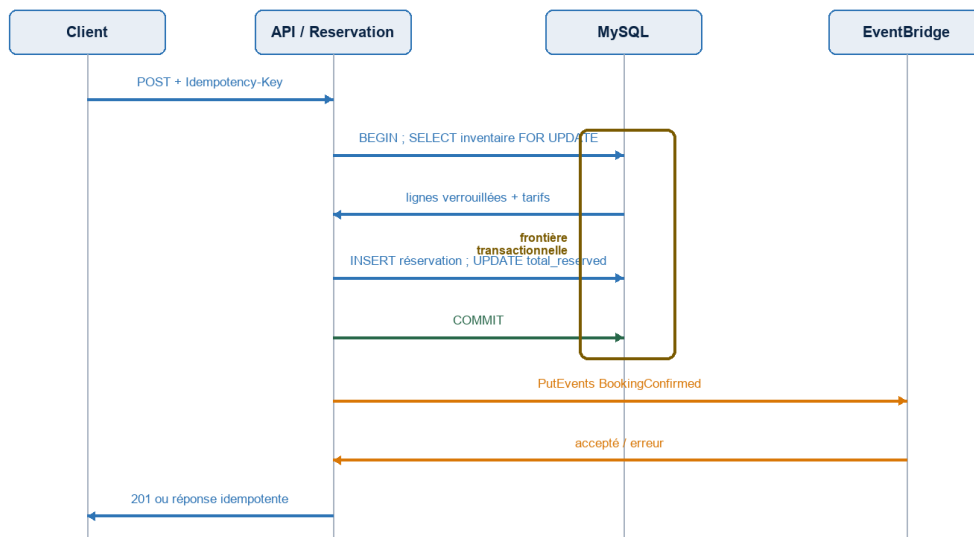


Figure 3 — Transaction de réservation et publication d'événement.

FOR UPDATE sérialise les réservations concurrentes portant sur les mêmes lignes d'inventaire. Le service verrouille toutes les nuits dans une transaction, vérifie la couverture et le minimum disponible, écrit la réservation, incrémente `total_reserved` puis commit. L'annulation verrouille la réservation, contrôle son état et décrémente avec `GREATEST`.

Pourquoi les stocks restent cohérents

Une deuxième transaction concurrente attend la libération des verrous, puis relit les compteurs déjà modifiés. Elle échoue si la capacité restante ne suffit plus. Rollback annule toutes les écritures en cas d'erreur.

Limites à traiter

La contrainte d'idempotence est globale alors que la recherche est scoppée par client : une collision inter-client ou deux premières requêtes strictement simultanées peut devenir un `duplicate-key/500`. Ajouter gestion explicite du conflit et tests d'intégration concurrents sur MySQL réel.

15

Événements, files et e-mails

Événement	Producteur	Consommateurs	Effet
BookingConfirmed	Reservation ECS	notification + analytics	E-mail et objet S3.
BookingCancelled	Reservation ECS	notification + analytics	E-mail et objet S3.
PartnerApplicationSubmitted	Reservation ECS	notification	Accusé de réception.
PartnerApplicationReviewed	Reservation ECS	notification	Décision approuvée/rejetée.

- Bus EventBridge dédié et archive 365 jours pour replay contrôlé.
- Une file notification et une file analytics, chacune chiffrée et associée à une DLQ ; maxReceiveCount = 3.
- Lambdas Node.js 24 hors VPC pour joindre SES/S3 sans NAT ; remontée des batchItemFailures message par message.
- Clés S3 analytiques déterministes afin qu'un retry écrase logiquement le même objet.

Atomicité incomplète

PutEvents intervient après COMMIT. Une panne peut laisser une réservation confirmée sans événement ; un replay idempotent peut aussi republier et doubler e-mail/analytics. Le modèle cible est une transactional outbox avec consommateur idempotent et déduplication.

Exploitation SES

Avant toute recette : vérifier l'expéditeur, sortir de sandbox si nécessaire, configurer DKIM/SPF/DMARC, suivre bounces/complaints et alerter sur la DLQ.

Preuves dans Le dépôt : `infra/Lib/events-stack.ts` ; `services/Lambda/notification-handler/index.js` ; `analytics-handler/index.js`

16

Analytique et Metabase

19. EventBridge duplique les événements de réservation vers la file analytique.
20. La Lambda normalise le payload et écrit un JSON sous année/mois/jour avec une clé déterministe.
21. Glue déclare la base et une table utilisant la projection de partitions.
22. Athena exécute les requêtes dans un workgroup dédié et écrit ses résultats dans un bucket séparé.
23. Metabase tourne dans ECS, utilise une base applicative dédiée initialisée par Lambda et accède à Athena via son rôle IAM.
24. Une distribution CloudFront et une HTTP API distinctes exposent l'interface Metabase via un listener ALB 8080.

Atout	Limite actuelle
Découplage du transactionnel	Seulement BookingConfirmed/Cancelled sont analysés.
Partitionnement sans crawler obligatoire	Format JSON moins compact que Parquet.
Permissions via rôle de tâche	Connexion Athena et dashboards à configurer manuellement.
Base Metabase séparée	Authentification Metabase indépendante de Cognito.
Service privé derrière edge	desiredCount 1 ; l'HTTP API directe peut contourner le WAF.

Évolution recommandée

Passer à Parquet, définir un schéma versionné et des métriques certifiées. Réduire ou pseudonymiser les PII dans les événements, borner la rétention après Glacier et remplacer les permissions Athena/Glue génériques par le moindre privilège.

Preuves dans Le dépôt : `infra/lib/analytics-stack.ts` ; `infra/lib/metabase-stack.ts` ; `services/Lambda/analytics-handler/index.js`

17

Identité, autorisation et protection

Couche	Contrôle
Identité utilisateur	Cognito User Pool, client SPA, confirmation e-mail, TOTP optionnel.
Rôles métier	Guest, HotelPartnerPending, HotelPartner, SuperAdmin.
API edge	Authorizer JWT sur routes protégées ; recherche publique ; CORS actuellement permissif (*).
Autorisation service	Contrôle groupe, statut partenaire et propriété hotel/réservation.
Secrets	Secrets Manager pour MySQL et Metabase ; injection par rôle d'exécution.
Données	Chiffrement RDS/OpenSearch/S3/SQS/ECR ; TLS et subnets privés.
Protection edge	WAF : rate-limit, Common Rule Set et SQLi Rule Set.
Audit	CloudTrail single-region + S3/Logs ; IAM Access Analyzer compte.

Risque de confiance réseau

Le middleware Reservation accepte les claims transmis par le chemin interne sans vérifier lui-même une signature JWT. Tant que les listeners restent ouverts au VPC, un appel interne peut contourner l'autorizer. Fermer l'ALB et valider cryptographiquement le jeton ou un mécanisme d'authentification de service.

- Activer rotation des secrets et procédures d'urgence.
- Ajouter GuardDuty, Security Hub, Inspector/ECR scanning et cdk-nag.
- Journaliser les décisions admin et protéger les données personnelles par politique RGPD.

Preuves dans Le dépôt : `infra/lib/auth-stack.ts` ; `frontend-stack.ts` ; `observability-stack.ts` ; `services/reservation-service/src/middleware/auth.js`

18

Frontend, UX et accessibilité

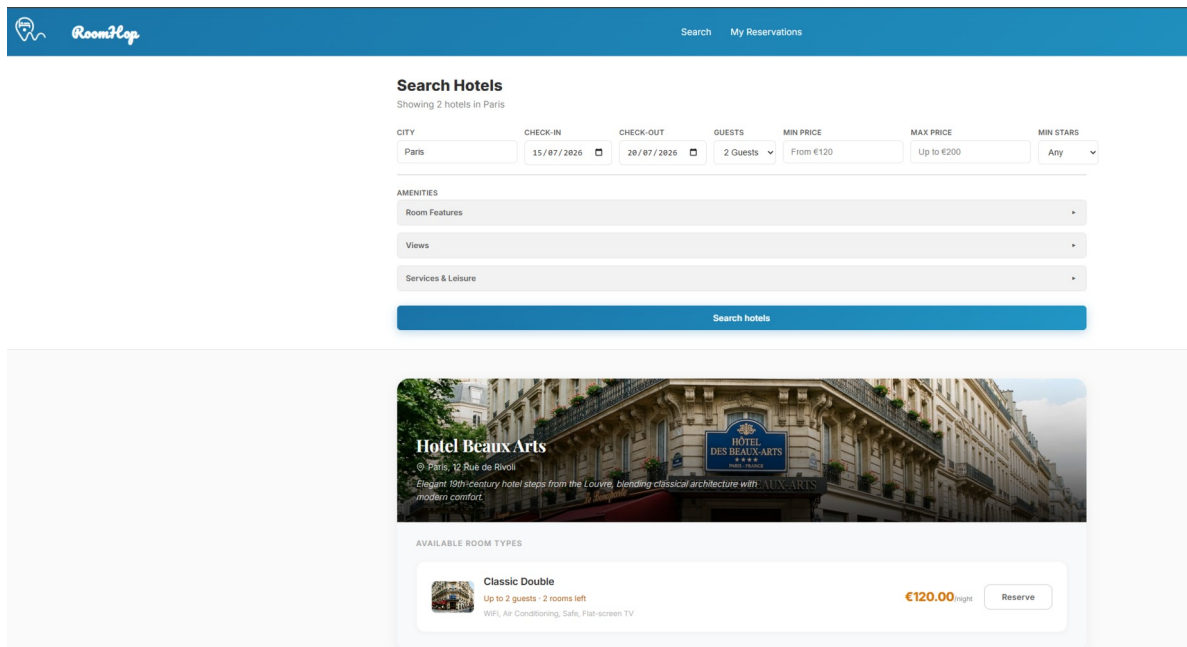


Figure 4 — Écran de recherche servant de référence visuelle RoomHop.

La SPA React/Vite applique une navigation publique et protégée. AuthContext centralise Cognito, useAuth évite les dépendances circulaires, le client HTTP ajoute le token et PrivateRoute oriente les statuts partenaire. CloudFront sert les routes SPA, l'API et les images sous une origine cohérente.

Onboarding propriétaire harmonisé

Le formulaire professionnel reprend la palette bleu/gris, les cartes, sections numérotées, espacements, états d'erreur/chargement, focus clavier, responsive et préférence reduced motion déjà présents dans recherche et authentification.

- Restent à traiter : dates dynamiques, filtre étoiles, cohérence du résumé tarifaire et écrans partenaire complets.
- Un audit WCAG global, des tests clavier/lecteur d'écran et des tests visuels navigateur sont encore requis.

Preuves dans Le dépôt : `hotel-ui/src/App.jsx` ; `auth` ; `pages` ; `PartnerOnboardingPage.module.css` ; `__tests__`

19

Infrastructure as Code et CI/CD

Étape	Service	Résultat
Source	GitHub via CodeConnections	Déclenchement sur branche paramétrée ; autorisation manuelle initiale.
Validate	CodeBuild	Tests, lint, build et synthèse CDK.
ContainerBuild	CodeBuild Docker	Images Search, Reservation, X-Ray et Metabase poussées dans ECR.
FrontendBuild	CodeBuild	Build Vite configuré avec outputs Cognito/API.
Deploy	ECS + S3 actions	Trois services mis à jour ; frontend copié dans S3.
Invalidate	CodeBuild	Invalidation CloudFront après livraison frontend.

CDK. Treize stacks TypeScript séparent Network, Database, OpenSearch, Auth, Events, Compute, Api, Frontend, Analytics, Metabase, DMS, Observability et Pipeline.

Premier déploiement. Les ECS tasks utilisent des Docker assets CDK ; elles ne dépendent pas d'images préexistantes dans ECR.

Limite du pipeline

La phase Validate synthétise l'infrastructure mais le pipeline ne lance pas cdk deploy. Il livre uniquement images ECS, assets S3 et invalidation. Les évolutions de stacks nécessitent donc un workflow d'infrastructure séparé, idéalement avec diff, approbation et déploiement contrôlé.

- Ajouter cdk diff, cdk-nag, scans dépendances/images et tests d'intégration.
- Prévoir comptes dev/stage/prod, approbation avant production et stratégie canary/blue-green.
- Versionner schémas et événements avec critères de compatibilité.

Preuves dans Le dépôt : `infra/lib/pipeline-stack.ts` ; `infra/bin/app.ts` ; `infra/package.json`

Observabilité et opérations

Signal	Présent	Manquant / amélioration
Logs applicatifs	Groupes CloudWatch explicites, logs structurés.	Corrélation uniforme et politique PII.
Conteneurs	Container Insights.	Alertes CPU/mémoire/restarts/targets.
Traçage	Daemon X-Ray sidecar.	Instrumentation SDK et propagation des segments.
Audit	CloudTrail management events vers S3 + Logs.	Trail multi-région, alertes sensibles et expiration des versions S3 non courantes.
Accès	IAM Access Analyzer.	Traitement formalisé des findings.
Asynchrone	DLQ et archive EventBridge.	Alarmes âge/profondeur/DLQ et runbook replay.
Données	Backup RDS 7 jours, Performance Insights.	Test restore, lag DMS, espace, connexions.
Expérience	Aucun SLO ni canary.	Synthetics, p95, taux d'erreur et parcours critique.

À ne pas surévaluer

La stack Observability ne crée ni dashboard ni alarmes métier. Le document considère donc l'observabilité comme un socle de télémétrie, pas comme une capacité d'exploitation aboutie.

- Définir SLI/SLO pour recherche, réservation et e-mail.
- Créer runbooks : stock conflictuel, DMS lag, OpenSearch indisponible, SES bounce, rollback ECS et restauration RDS.
- Tester périodiquement alertes, replay et restauration.

Preuves dans Le dépôt : infra/lib/observability-stack.ts ; Logs dans compute/events/database/metabase ; aucun AWS::CloudWatch::Alarm dans La synthèse

21

Validation et préparation au déploiement

Validation	Résultat observé
Backend et services	65/65 tests réussis.
Frontend	10/10 tests, lint et build réussis.
Infrastructure	5/5 tests d'architecture, TypeScript et synthèse de 13 stacks réussis.
Conteneurs	4/4 images construites localement.
AWS	Aucun déploiement ; Cognito, SES, DMS, OpenSearch, réseau et pipeline non testés intégrés.

Checklist avant une première recette éphémère :

- Définir compte/région, bootstrap CDK et tags ; examiner cdk diff et coûts.
- Fermer les listeners ALB, renforcer JWT/validation serveur et corriger dates/filtre étoiles.
- Vérifier SesSenderEmail, destinataires sandbox, DKIM et configuration des événements.
- Autoriser CodeConnections GitHub et décider qui déploie réellement les stacks.
- Charger les images, initialiser Metabase, connecter Athena et restreindre son accès.
- Exécuter tests E2E, concurrence MySQL réelle, panne OpenSearch, DLQ/replay et restauration.
- Créer Budget/Anomaly Detection et une procédure de destruction ; contrôler les ressources RETAIN.

Commande de synthèse

cd infra ; npm ci ; npm run build ; npm test ; npm run synth. La synthèse produit des templates ; elle ne crée aucune ressource AWS.

Preuves dans Le dépôt : résultats de validation du projet ; infra/README.md ; package.json des composants

22

Modèle de coût théorique

Ordre de grandeur à la demande, us-east-1, daté du 5 septembre 2026

Poste fixe	Taux estimé	Part
11 endpoints Interface dans 2 AZ	0.2200 USD/h	44.7 %
Fargate : Search + Reservation + Metabase	0.0987 USD/h	20.1 %
RDS MySQL db.t3.medium Single-AZ	0.0680 USD/h	13.8 %
OpenSearch t3.small.search, 1 nœud	0.0360 USD/h	7.3 %
DMS dms.t3.micro	0.0360 USD/h	7.3 %
ALB interne, composante horaire	0.0225 USD/h	4.6 %
Stockage provisionné agrégé, ordre de grandeur	0.0108 USD/h	2.2 %

Durée	Socle fixe	Avec faible trafic/logs
4 heures	1.97 USD	≈ 2,03 USD
5 heures	2.46 USD	≈ 2,52 USD
1 exécution pipeline	hors socle	≈ +0,39 USD

Lecture correcte

Estimation brute hors taxes, transferts importants, requêtes, logs volumineux et variations tarifaires. Les endpoints privés dominent le socle. OpenSearch en RemovalPolicy RETAIN peut continuer à coûter environ 0,94 USD/jour s'il est oublié après destruction.

Référence : [AWS Pricing Calculator](#)

23

Évaluation Well-Architected — méthode et synthèse

Cette évaluation interne confronte le code, les templates synthétisés et les tests aux six piliers AWS. Elle n'est pas une revue officielle réalisée dans AWS Well-Architected Tool. En l'absence de déploiement, elle évalue l'intention et les garde-fous, pas les données d'exploitation.

Maturité Well-Architected indicative — échelle 1 à 5

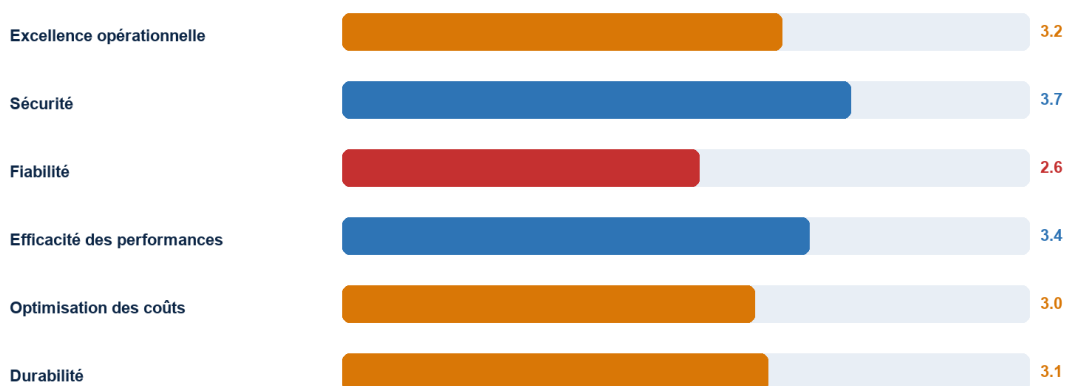


Figure 5 — Score de maturité indicatif par pilier.

Échelle	Interprétation
1	Risque élevé ou absence de mécanisme.
2	Socle partiel ; lacunes structurantes.
3	Conception cohérente ; exploitation à compléter.
4	Pratique solide, mesurée et automatisée.
5	Optimisation continue démontrée.

Résultat global

Moyenne indicative 3.2/5. La sécurité et la performance ont un bon socle ; la fiabilité est volontairement limitée par les choix mono-instance.

24

Well-Architected — excellence opérationnelle et sécurité

Pilier	Forces	Écarts prioritaires	Score
Excellence opérationnelle	CDK, 13 stacks, tests, pipeline V2, logs explicites, CloudTrail, DLQ/archive.	Pas de déploiement infra dans pipeline, dashboard/alarme/SLO/runbooks ; X-Ray non instrumenté ; documentation locale divergente.	3,2/5
Sécurité	Cognito, groupes, JWT edge, propriété métier, WAF, chiffrement, secrets, réseau isolé, Access Analyzer.	Listeners ALB ouverts au VPC, confiance claims interne, rotation/GuardDuty/Security Hub/cdk-nag/PII à ajouter.	3,7/5

Appréciation détaillée — opérations

- Prepare : infrastructure et tests automatisés, mais readiness review, runbooks et RTO/RPO non formalisés.
- Operate : télémétrie créée, mais aucune alarme ni procédure de triage codée.
- Evolve : pipeline favorise les changements applicatifs ; le chemin d'évolution des stacks reste manuel.

Appréciation détaillée — sécurité

- Identité forte au périmètre utilisateur et contrôles d'appartenance côté service.
- Défense en profondeur incomplète entre API Gateway et ALB tant que open: false et validation cryptographique ne sont pas appliqués.
- Protection des données convenable par défaut ; gouvernance, rotation et réponse à incident à industrialiser.

[AWS — Operational Excellence Pillar](#)

[AWS — Security Pillar](#)

25

Well-Architected — fiabilité et efficacité des performances

Pilier	Forces	Écarts prioritaires	Score
Fiabilité	Transactions, idempotence, fallback, autoscaling, SQS/DLQ, archive, sauvegarde et IaC.	RDS Single-AZ, desiredCount 1, OpenSearch 1 nœud, DMS Single-AZ, pas d'outbox ni tests de reprise/charge.	2,6/5
Performance	CloudFront, OpenSearch, indexes SQL, services stateless, autoscaling 1-4, traitement asynchrone.	Aucune mesure p95/charge, assemblage multi-index, résultats vides non contrôlés, tailles non benchmarkées.	3,4/5

Fiabilité sous contraintes

Le maintien volontaire de Single-AZ et desiredCount 1 est compatible avec une démonstration de quelques heures, pas avec un objectif de haute disponibilité. Tant que ces invariants restent imposés, les compensations sont : backups, restore drill, fallback testé, redeploy rapide, alerte et communication d'indisponibilité.

- Définir RTO/RPO et tester perte RDS, panne OpenSearch, arrêt task ECS, lag DMS et DLQ.
- Ajouter transactional outbox et déduplication pour la chaîne après commit.
- Mesurer recherche/réservation p50/p95, taux 4xx/5xx, connexions DB, CPU/mémoire, cache CloudFront et lag.
- Benchmark avant redimensionnement ; tester montée 1→4 tasks et limites MySQL.

[AWS — Reliability Pillar](#)

[AWS — Performance Efficiency Pillar](#)

26

Well-Architected — optimisation des coûts et durabilité

Pilier	Forces	Écarts prioritaires	Score
Optimisation des coûts	Pas de NAT, petites tailles, desiredCount 1, autoscaling, services managés et estimation éphémère.	22 endpoints-AZ, services always-on, aucun budget/tagging/plan d'arrêt, RETAIN OpenSearch et droitsizing non mesuré.	3,0/5
Durabilité	Services managés, CDN, asynchronisme, petites capacités et absence de NAT.	Aucun KPI d'impact, cycle de vie data, droitsizing réel, Graviton ni suppression automatique des environnements.	3,1/5

- Créer tags de coûts obligatoires, AWS Budget, Anomaly Detection et alerte avant une démonstration.
- Évaluer si les 11 endpoints doivent tous exister dans deux AZ pour un environnement court ; documenter le compromis coût/résilience.
- Automatiser arrêt/destruction et inventaire des ressources RETAIN ; vérifier après cdk destroy.
- Mesurer coût, vCPU-seconde, Gio-stockés et données transférées par recherche/réservation.
- Compacter l'analytics en Parquet, appliquer lifecycle et supprimer les données de test.
- Comparer Fargate ARM64/Graviton et versions logicielles plus efficaces après benchmark.

Compromis central

Les endpoints privés suppriment le NAT et renforcent le contrôle réseau, mais constituent environ 45 % du socle estimé. La bonne décision dépend de la durée, du niveau d'isolation et du besoin multi-AZ.

[AWS — Cost Optimization Pillar](#)

[AWS — Sustainability Pillar](#)

27

Feuille de route d'amélioration

Horizon	Actions	Valeur
P0 — avant déploiement	Fermer listeners ALB ; validation JWT/service ; dates dynamiques ; filtre étoiles ; validation serveur stricte ; contraintes DB.	Sécurité et exactitude produit.
P0 — fiabilité métier	Transactional outbox, déduplication consommateurs, gestion duplicate idempotency, tests MySQL concurrents.	Éviter événement perdu/doublé et erreurs 500.
P0 — exploitation	Alarmes, dashboard, SLO, logs corrélés, DLQ/replay, DMS lag, SES bounce/complaint et runbooks.	Détecter et restaurer rapidement.
P0 — gouvernance	Tags, Budget, Anomaly Detection, procédure destroy/RETAIN, secrets et PII.	Limiter coût et risque.
P1 — expérience	Montant exact avant confirmation, room_count, demandes spéciales, gestion partenaire complète, file SuperAdmin, upload images.	Finir les parcours.
P1 — recherche	Fraîcheur d'index, fallback sur incohérence, Support LOB DMS, starter onUpdate, alarmes de lag.	Réduire faux zéros.
P1 — livraison	Pipeline infra séparé avec diff/approbation, scans, comptes dev/stage/prod et E2E.	Changements contrôlés.
P2 — production	Si contraintes levées : RDS Multi-AZ, desiredCount ≥2, OpenSearch multi-nœud/zone, stratégie DR.	Haute disponibilité.
P2 — data	Parquet, métriques certifiées, lifecycle, privacy retention et dashboards préconfigurés.	BI économique et gouvernée.

Ordre recommandé

Corriger d'abord les frontières de confiance et l'atomicité. Ajouter ensuite mesure et tests intégrés. Les changements de haute disponibilité restent conditionnels : ils ne modifient pas les deux invariants explicitement conservés pour la version actuelle.

Definition of Done P0 : synthèse propre, tests verts, aucune route interne contournable, réservation/événement récupérables, alertes testées et destruction contrôlée.

Registre des risques

ID	Risque	Prob.	Impact	Traitement
R-01	Listener ALB ouvert par défaut	Élevé	Élevé	open:false + ingress SG VPC Link
R-02	Claims internes non vérifiés cryptographiquement	Moyen	Élevé	JWT/service auth + réseau fermé
R-03	Commit sans événement ou doublon au replay	Moyen	Élevé	Outbox + déduplication
R-04	OpenSearch vide mais index incomplet	Moyen	Moyen	Fraîcheur + fallback contrôlé
R-05	RDS Single-AZ indisponible	Moyen	Élevé	Backup, restore drill, RTO/RPO ; Multi-AZ en prod
R-06	Une seule task applicative	Moyen	Moyen	Auto-remplacement ; desired≥2 en prod
R-07	OpenSearch/DMS mono-instance	Moyen	Moyen	Fallback, alarmes, procédure reload
R-08	SES sandbox/identité non prête	Élevé	Moyen	Recette SES et suivi DLQ
R-09	Pipeline ne déploie pas l'IaC	Élevé	Moyen	Workflow infra dédié
R-10	Divergence local/AWS et README obsolète	Élevé	Faible	Docs générées, tests contractuels
R-11	Validation réservation permissive	Moyen	Élevé	Schémas stricts + contraintes SQL
R-12	PII/RGPD non gouverné	Moyen	Élevé	Classification, rétention, droits d'accès

Risque résiduel accepté

R-05 et R-06 ne sont pas supprimés dans la version actuelle : Single-AZ et desiredCount 1 sont des contraintes volontaires. Leur acceptation doit être limitée à un environnement de démonstration sans engagement de disponibilité.

29

Traçabilité et couverture de tests

Domaine	Comportements couverts	Preuve	Limite
Recherche	Paramètres, prix, disponibilité, fallback	5 tests service + propriétés API	Unitaire/contractuel
Réservation	Montant, stock, idempotence, propriété	8 tests service + propriétés API	Mocks ; MySQL réel absent
Partenaire	Identité, validation, statut, événements	Tests admin + 2 tests UI	Service/UI
Notification	4 modèles, parsing, SES, batch failures	5 Lambda + 6 local	Client SES simulé
Analytics	Normalisation et clé S3	2 tests	Client S3 simulé
Migration	Plan bootstrap/incréments/DELETE	4 tests	Pas de RDS réel
Metabase init	Création DB/utilisateur	1 test	Connexion simulée
Frontend	Formulaire et idempotency hook	10 tests + lint/build	Pas d'E2E navigateur final
CDK	Invariants réseau/RDS/ECS/Lambda	5 tests + tsc + synth	13 stacks, 270 ressources
Docker	Build images	4/4	Pas de scan/registry

Bilan

65 tests backend/services et 10 tests frontend réussissent. Cette couverture réduit le risque de régression locale, mais ne valide ni IAM, ni réseau, ni intégrations managées, ni performance, ni récupération.

- Ajouter tests de concurrence contre MySQL 8 réel.
- Ajouter E2E Cognito→API→ECS→RDS et EventBridge→SQS→Lambda→SES.
- Ajouter tests de panne, charge, accessibilité et sécurité des frontières.

Preuves dans Le dépôt : `hotel-api/__tests__` ; `hotel-ui/__tests__` ; `services/*/test` ; `services/Lambda/*/test` ; `infra/test/architecture.test.ts`

Glossaire et références

Terme	Définition
AZ	Availability Zone, domaine de défaillance isolé dans une région.
CDC	Change Data Capture, réplication des changements depuis les binlogs.
CDK	Cloud Development Kit, infrastructure décrite en TypeScript.
DLQ	Dead-letter queue, file recevant les messages après échecs répétés.
Idempotence	Même demande répétée : un seul effet métier.
OAC	Origin Access Control, accès CloudFront à un bucket S3 privé.
RPO / RTO	Perte de données tolérée / délai de restauration toléré.
SLO	Objectif mesurable de niveau de service.
SigV4	Signature AWS des requêtes API.
Transactional outbox	Événement écrit avec la transaction puis publié de façon fiable.

Références AWS officielles

[AWS Well-Architected — vue d'ensemble](#)

[Operational Excellence Pillar](#)

[Security Pillar](#)

[Reliability Pillar](#)

[Performance Efficiency Pillar](#)

[Cost Optimization Pillar](#)

[Sustainability Pillar](#)

[AWS Pricing Calculator](#)

Références projet principales

README.md ; infra/README.md ; infra/bin/app.ts ; infra/lib/*.ts ; infra/test/architecture.test.ts ; database/migration_v2.sql ; hotel-ui/src ; services/search-service ; services/reservation-service ; services/lambda.

Fin du document

La prochaine révision doit intégrer les résultats d'un déploiement de recette, les mesures de charge, les preuves SES/DMS et les décisions prises sur les améliorations P0.